



Name: William Kojumian

Bachelor of Engineering Science
American University of Armenia

Supervisor: Krist Samuel

Capstone Project: Drone System for Intralogistical Payload Transport

May 21, 2025

Contents

1	Introduction	3
1.1	Project Description	3
1.1.1	Problem Statement	3
1.1.2	Background	5
1.1.3	Assumptions and Constraints	5
1.2	Overview of the Envisioned System	6
1.2.1	Overview	6
1.2.2	Components	7
1.2.3	Drone Frame Material	8
1.2.4	Critical Hardware Components	9
1.2.5	Circuit Diagram	10
1.2.6	Beta Version Drone	11
1.2.7	System Scope	12
1.3	Drone Weight and Payload Calculations	12
2	Documents	13
2.1	Applicable Documents	13
2.2	Reference Documents	14
3	Description of Envisioned System	15
3.1	Needs, Goals and Objectives	15
3.2	Overview of System and Key Elements	15
3.2.1	System Modeling	15
3.3	Thrust per motor	16
3.3.1	Altitude Regulation	16
3.3.2	Motor Speed Relationship	16
3.3.3	System Control	17
3.4	System Parameters	17

3.5	Interfaces	21
3.5.1	ArUco Marker Detection	21
3.5.2	Servo Actuator Control	28
3.6	Modes of Operations	34
3.7	Proposed Capabilities	35
3.8	Methodology	36
4	Physical Environment	38
5	Support Environment	39
6	Risks and Potential Issues	40

1 Introduction

1.1 Project Description

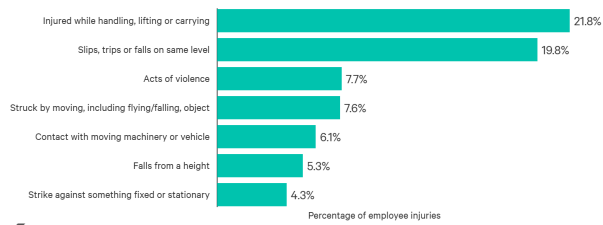
1.1.1 Problem Statement

Warehouse environments are harsh. There are many ways in which labor in a warehouse environment can be unsophisticated, such as when forklift operators accidentally injure themselves when dealing with heavy machinery or, when inventory clerks have to position themselves in very elevated altitudes and accidentally slip and harm themselves or when warehouse associates have to walk for very long distances, which takes time, such as going to the bathroom during their recess time already took up most of their time, thus making it an exhausting experience. Data from health and safety executive reports show that handling, lifting, or carrying accounts for 20% of non-fatal injuries in the workplace. In addition, there is data from the Reporting of Injuries, Diseases, and Dangerous Occurrence Regulations and the Labor Force Survey, which also has data from workplace inconveniences in the warehouse environment.

As we can see in the diagrams below, the reports show significant injuries, demonstrating how prominent this issue is. We can observe the many different reasons for those injuries, and based on the diagrams, we can see that most of these injuries are caused by employees dealing with heavy payloads, dangerous machines, using ladders, which then position themselves in hazardous locations, etcetera. Therefore, I want to develop a project for my engineering capstone; the project is based on a drone system that can autonomously grapple onto a payload and adjust its DC motor speed using control theory to be stable in the air, navigate its way around the warehouse using Aruco markers, pinpoint specifically place the crate onto the rack where it's usually dangerous for manual labor to do, and return to its location again. With the help of this drone system, we are eliminating and mitigating most of the dangers discussed earlier. Developing this technology will lessen the gap in our modern approach to dealing with oil-dependent heavy machinery that requires manual labor; with this drone, we are shifting to a more environmentally friendly approach.

7 Most Common Kinds of Non-Fatal Workplace Injuries

Lifting/carrying and slips/trips/falls account for almost half of workplace injuries



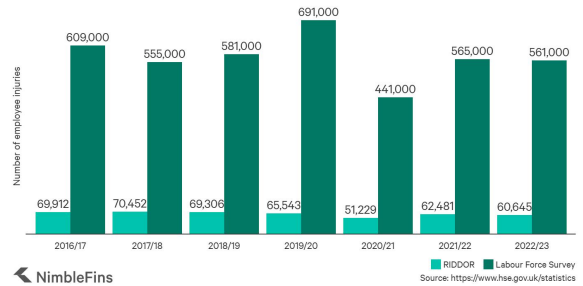
NimbleFins

Source: NimbleFins analysis of Labor Force Survey and RIDDOR data from <https://www.hse.gov.uk/statistics>

(a) Common Kinds of Injuries

Number of Non-fatal Workplace Injuries to Employees in Great Britain

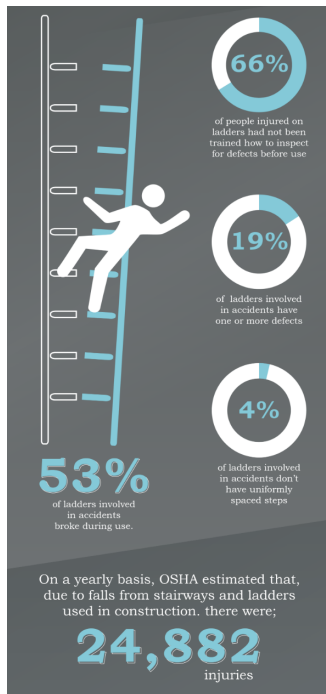
Labour Force Survey data reflects self-reported injuries and RIDDOR data reflects company-reported injuries



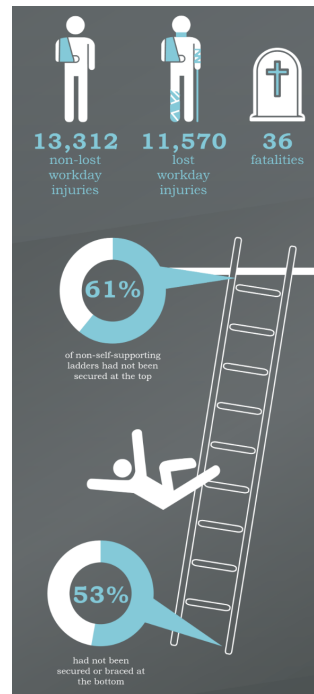
NimbleFins

Source: <https://www.hse.gov.uk/statistics>

(b) Number of Injuries to Employees



(c) Ladder Injuries Analysis



(d) Ladder Injuries Results

Figure 1: Visualization of workplace injury statistics

1.1.2 Background

Drones, in general, are used in various fields and for different reasons. They can be helpful in many aspects, such as videography, security, environmental monitoring, mapping, or even entertainment. On October 13, 2023, Amazon released information that they would start expanding their drone delivery sector in many states in the US and UK. The idea of drone delivery is fascinating to me, and I see much potential in this sector for the future. Even research claims that drones are more cost effective than traditional express delivery methods, costing around \$1.23 per delivery compared to \$5.33 for a four-mile distance by electric van. Moreover, they are fully electric, thus contributing to a more environmentally friendly approach.

However, it is essential to consider why this technology is not advanced enough to be widely used worldwide. For instance, weather conditions such as rain or extreme heat can cause drones to malfunction, leading them to crash and potentially damage property or even injure people below, which should be avoided at all costs. Moreover, there is a legal aspect to consider; air traffic regulations exist, and airports and airplanes occupy regions where airspace is taken up. Allowing drones to fly around these areas would pose significant aerodynamic risks. As a result, current laws prevent the widespread use of drone delivery systems. Given these challenges, I wanted to approach my project more practically, focusing on indoor environments like warehouses, where weather and air traffic have no impact or interference.

1.1.3 Assumptions and Constraints

Of course, in a warehouse environment, loads are heavy, up to 23 kilograms. Thus, my project will be a prototype yet still follow the same aerodynamic, physics, and control theoretical approach. The prototype drone I will build will hold 1-2 kilograms of payload while maintaining stability. Furthermore, there is a sacrifice to increase safety and decrease labor; thus, there are some constraints, such as the proper limit mentioned earlier. Moreover, another couple of constraints could be that the drone will not last very long with a 5000mAh Lipo Battery; assuming the drone will run its motors at throttle max capacity, with the

calculations, it will last for around 6 minutes, which is decent for organizing crates in a warehouse, but recharging it and regulation will be necessary, of course, unless the drone is even more advanced to auto charge itself. One last constraint is that a drone with a speeding propeller can be dangerous. Employees should be aware of the fact not to get close to the drone, or it can cause severe damage, and of course, make sure the drone is at the correct altitude and nothing to interrupt the propellers as it flies because injuries and damage will most likely result if the propellers were spinning at max speed then gets disconnected suddenly. Ultimately, the project I will be building mainly focuses on the highest rack; for the drone to place a payload in a pay rack somewhere in the middle would be challenging and risky for my project since it requires immense precision and space. Finally, I am using a GPS module, but GPS might not function properly since it is in an indoor warehouse.

Battery Capacity: 5000 mAh

Consumption at full throttle for all 4 motors with weight = $12\text{ A} \times 4 = 48\text{ A}$

Flight Time:

$$\text{Flight Time} = \frac{5000\text{ mAh}}{48\text{ A}} = 0.1\text{ hours} = 6\text{ minutes}$$

1.2 Overview of the Envisioned System

1.2.1 Overview

In summary, the warehouse operator has to slide the crate directly under the drone before it turns on, then once the drone is switched on, it latches onto the crate using the 3D-printed mechanism and SG90 servos within it. Then, using the MTF01-based flow sensors, it will ascend until it reaches and maintains a certain altitude, ensuring it does not hit the ceiling and remains higher than the other racks. With its GPS, the drone will go towards its destined location while having a rough estimate until it detects the ArUco marker, which it intends to use as a landing pad and position the payload accurately at the designated location. Finally, the drone will reverse its movements and return to its original position. Therefore, this drone aims to transport a payload from point A to point B, where point B is typically a challenging location, reducing the risk of labor injuries. The components used in this project include a drone frame, DC motors, electronic speed controllers, propellers, a flight control shield, a

processing unit, a camera, ArUco markers, servo motors, a battery, an ultrasonic sensor, a power distribution board, and other fundamental tools such as a soldering kit, multimeter, and screwdriver.

1.2.2 Components

Component	Description and Notes
Tarot 650 (Frame)	Carbon fiber quadcopter frame (650mm motor-to-motor) providing structural integrity with minimal weight.
Tarot 4108 (380Kv) Motors (x4)	High-torque brushless motors optimized for multirotors, with 380Kv rating.
T-Motor FLAME 40A ESCs (x4)	Precision ESCs with thermal management, supporting 6S LiPo.
T-Motor CF 1355 Propellers (x4)	13.55-inch carbon fiber props for efficient thrust generation.
GEPRC Storm 6S 5000mAh 95C Battery	High-current LiPo battery (95C discharge) for demanding maneuvers.
Pixhawk 4 (Flight Controller)	Redundant ARM Cortex-based flight controller.
Raspberry Pi 4 2GB (SBC)	Main computing unit running ROS and vision pipelines.

Table 1: Primary Flight Hardware Components

Component	Description and Notes
Raspberry Pi HQ Camera	12.3MP Sony IMX477 sensor for computer vision.
Dynamixel AX-12A Servo Motors	Smart servos with feedback control for gripping.
TL2996 (PDB)	Power distribution board with voltage regulation.
Neo-M8N GPS Module	Multi-constellation GNSS receiver.
MASF01-based Flow Sensor	Optical flow sensor for GPS-denied navigation.
LM2596 Voltage Dropper	Buck converter for 5V/3.3V subsystems.
3D Printed Gripping Mechanism	Lightweight custom end-effector.
3D Printed Camera Protection	Vibration-isolated camera housing.
3D Printed GPS Positioner	Optimized antenna mount.

Table 2: Vision, Payload, and Support Components

1.2.3 Drone Frame Material

The drone frame is essential for the drone, especially when every drone has its purpose. In this case, the drone frame must have maximum sturdiness and a low weight ratio. I used material science knowledge and went to EduPack software to check the ideal material for tensile strength, with good maximum service temperature to handle the temperature of the sun and hot components, as well as fracture toughness to not crack and shatter in case of any external sudden forces. With all these constraints and limits added, we end up with this density vs. young modulus graph, ensuring that carbon fiber is the most viable option as a composite material.

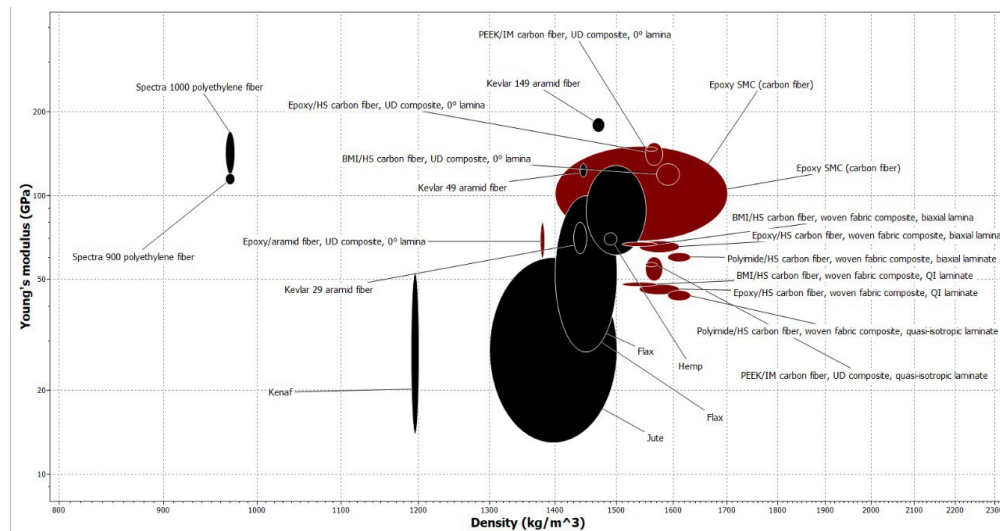


Figure 2: Density vs. Young Modulus Graph after setting specific constraints Dark Red Circles Resemble Composite Materials resulting in Carbon Fiber

1.2.4 Critical Hardware Components

The system relies on two essential hardware components, each playing a vital role in the drone's operation:



(a) Raspberry Pi 4



(b) Pixhawk 4

Raspberry Pi 4 (Fig. 3a): Serves as the central computing unit, handling high-level tasks including:

- Real-time image processing
- Communication with ground control
- Path planning algorithms
- Sensor data fusion

Pixhawk 4 (Fig. 3b): The flight controller responsible for:

- Low-level flight stabilization
- Motor control and ESC communication
- Inertial measurement processing
- Fail-safe mechanisms

1.2.5 Circuit Diagram

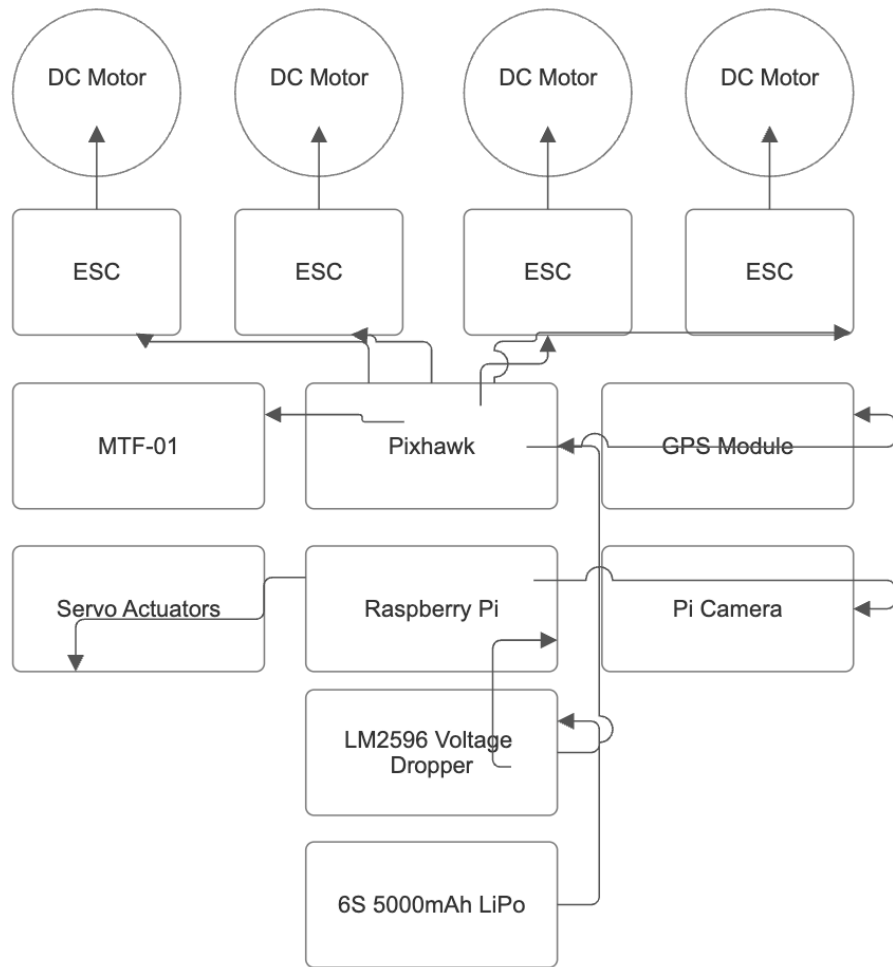


Figure 4: Complete wiring diagram of the drone system showing all electrical connections and components

1.2.6 Beta Version Drone



Figure 5: Real world photo of the unpolished beta version of the drone system showing all electrical connections and components

1.2.7 System Scope

I mostly buy the pieces for the drone online, meaning that the frame is pre-built and not 3D printed by me. The goal of this project is quite a challenge. I will be connecting all the drone parts together, adding the code for the drone to perform the autonomous task, and integrating the control system to ensure the drone acts stably. Of course, the drone can transport the load across the warehouse, but this project will focus on a nearby rack for a more presentable approach.

The type of payload it will carry will be a simple box, ensuring that the inside is not interfered with – a perfectly cubical hollow. However, the outside may be adjusted to enhance the grappling system, preventing the load from slipping from the drone. Even though I mentioned that this project is a prototype, we can consider the system’s maximum payload based on physics:

1.3 Drone Weight and Payload Calculations

Total Thrust:

$$\text{Total Thrust} = 1 \text{ kg per motor} \cdot 4 = 4 \text{ kg}$$

Drone Weight:

$$\text{Drone Weight} = 300 \text{ g (frame)} + 220 \text{ g (motors)} + 120 \text{ g (ESCs)} + 40 \text{ g (propellers)} + 500 \text{ g (battery)} +$$

$$400 \text{ g (miscellaneous)} = 1580 \text{ g} = 1.58 \text{ kg}$$

Max Payload:

$$\text{Max Payload} = \text{Total Thrust} - \text{Drone Weight} = 4 \text{ kg} - 1.58 \text{ kg} = 2.42 \text{ kg}$$

2 Documents

2.1 Applicable Documents

There are documents from the Federal Aviation Administration Rules and Regulations for Unmanned Aircraft Systems (UAS), which state that their main concern is safety.

Hobby or Recreation (Section 336)	Commercial Use (Part 107)
» Flying a model aircraft at the local model aircraft club » Taking photographs with a model aircraft for personal use » Using a model aircraft to move a box from point to point without any kind of compensation » Viewing a field to determine whether crops need water when they are grown for personal enjoyment	» Receiving money for demonstrating aerobatics with a model aircraft » A realtor using a model aircraft to photograph a property that he is trying to sell and using the photos in the property's real estate listing. » A person photographing a property or event and selling the photos to someone else. » Delivering packages to people for a fee » Determining whether crops need to be watered that are grown as part of commercial farming operations.

Figure 6: FAA Rules and Regulations for UAS

2.2 Reference Documents

For this CONOPS, I used the following articles and documents to help in researching and understanding my project approach:

- **Federal Aviation Administration Laws:** https://www.faa.gov/uas/advanced_operations/package_delivery_drone
- **Amazon Drone:** <https://www.aboutamazon.com/news/transportation/amazon-drone-delivery-arizona>
- **Pros and Cons of Drone Delivery:** <https://theweek.com/tech/drone-delivery-pros-cons>
- **When Amazon announced their first drone delivery:** https://en.wikipedia.org/wiki/Amazon_Prime_Air#:~:text=In%20October%202023%2C%20Amazon%20announced,and%20then%20expanding%20over%20time.
- **Warehouse Incident Statistics (Thompsons Law):** <https://www.thompsons.law/support/legal-guides/common-warehouse-accidents-and-how-to-avoid-them>
- **Warehouse Incident Statistics (Falcony):** <https://blog.falcony.io/en/15-most-common-safety-incidents-in-warehousing-and-distribution>
- **Warehouse Incident Statistics (Ludus Global):** https://www.ludusglobal.com/en/blog/serious-accidents-in-logistics-warehouses?hs_amp=true
- **Warehouse Incident Statistics (WSS EU):** <https://wss-eu.com/blog/5-common-warehouse-injuries-and-strategies-to-avoid-them/>
- **Warehouse Incident Statistics (Nimblefins):** <https://www.nimblefins.co.uk/business-insurance/employers-liability-insurance-uk/workplace-injury-statistics-uk#number>
- **Drone Delivery Advantages:** <https://www.sciencedirect.com/science/article/abs/pii/S1361920923002286#:~:text=Drone%20deliveries%20significantly%20enhance%20accessibility,Tavares%20%26%20Brussels%2C%202019>

3 Description of Envisioned System

3.1 Needs, Goals and Objectives

Warehouses have a prominent issue where almost 2 out of 10 machinery operators, or nearly any employee dealing with organizing the intralogistical system, get injured. Sometimes, boxes aren't effectively organized, which may cause them to fall off the rack and lead to damage. Oil leaks can occur from tractors, which may result in people slipping, or in the worst case, the oil could be reactive or flammable. Therefore, it is safe to say that the real world needs advancements in warehouse operations.

This project aims to develop a concept for autonomous drones that can assist in improving the challenging working environment faced by warehouse employees. The aim is to reduce human risk and increase efficiency. In addition, I plan to learn and share the components, code, and approaches used to design the drone, including a unique and suitable approach to navigation and a grappling system that takes automation to the next level.

3.2 Overview of System and Key Elements

Since I am applying control theory to the system, a block diagram was designed:

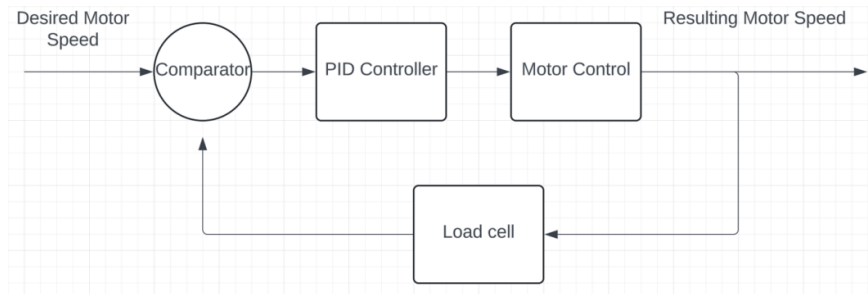


Figure 7: Block Diagram of the Control System

3.2.1 System Modeling

The system dynamics of the drone can be represented by the following relationships:

3.3 Thrust per motor

$$T = k_{\text{thrust}} \cdot \omega^2 \quad (1)$$

Where k_{thrust} is the thrust constant and ω is the motor speed (in radians per second).

3.3.1 Altitude Regulation

The total thrust produced by the four motors must balance the gravitational force on the drone and payload:

$$\sum T = m \cdot g \quad (2)$$

Where m is the mass of the drone and payload, and g is the gravitational constant. The sum of the thrust from all four motors must balance the gravitational force to maintain stable altitude.

3.3.2 Motor Speed Relationship

The motor speed is a function of the applied voltage to the motor, which can be modeled by the following transfer function:

$$G(s) = \frac{\omega(s)}{V(s)} = \frac{K_m}{\tau s + 1} \quad (3)$$

Where:

- K_m is the motor constant (in $\text{V} \cdot \text{s}/\text{rad}$),
- τ is the motor time constant (in seconds),
- s is the Laplace variable,
- $\omega(s)$ is the motor speed in the Laplace domain,
- $V(s)$ is the applied voltage in the Laplace domain.

3.3.3 System Control

To regulate the altitude, a PID controller is implemented within the feedback loop. The controller adjusts the voltage applied to the motors to minimize the error between the desired and actual altitude. The PID control law is expressed as:

$$P(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot \frac{d}{dt}e(t) \quad (4)$$

Where:

- $P(t)$ is the control output (voltage),
- $e(t) = h_{\text{desired}}(t) - h(t)$ is the error signal, representing the difference between the desired altitude and the actual altitude,
- K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively.

The controller adjusts the motor voltages based on the altitude error, which is then converted into motor speeds to maintain the desired altitude.

3.4 System Parameters

For a medium-quality drone with brushless DC motors and a 1 kg payload, the following values have been chosen:

- **Motor constant:** $K_m = 0.5 \text{ V} \cdot \text{s/rad}$
- **Motor time constant:** $\tau = 0.2 \text{ seconds}$
- **Thrust constant:** $k_{\text{thrust}} = 0.02 \text{ N/s}^2$
- **Mass of drone + payload:** $m = 2.5 \text{ g}$
- **Acceleration due to gravity:** $g = 9.81 \text{ m/s}^2$

The closed-loop transfer function is derived by combining the motor dynamics with the PID controller. The resulting system is given by:

$$H(s) = \frac{k_m \cdot k_{\text{thrust}}}{\tau s + 1} \cdot \frac{1}{m} \quad (5)$$

The PID controller is applied to this system using feedback to regulate the altitude.

Root Locus Code:

```
Km = 0.5;
tau = 0.2;
kthrust = 0.02;
m = 2.5;
g = 9.81;
s = tf('s');
G_motor = Km / (tau * s + 1);
G_thrust = kthrust;
H = G_motor * G_thrust / m;
Kp = 5;
Ki = 1;
Kd = 0.1;
C = pid(Kp, Ki, Kd);
sys = feedback(C * H, 1);
figure;
rlocus(sys);
title('Root Locus of the Closed-Loop System');
```

Root Locus:

I tried applying the Ziegler-Nichols method to determine the appropriate PID gains. This approach involves experimentally determining the critical gain (K_c) and critical period (P_c) from the open-loop response of the system. With these values, I used them to calculate the gains for the PID respectively. The Ziegler-Nichols tuning rules are as follows:

- The proportional gain K_p is calculated as $K_p = 0.6 \times K_c$.
- The integral gain K_i is computed using $K_i = \frac{P_c}{2} \times K_p$.

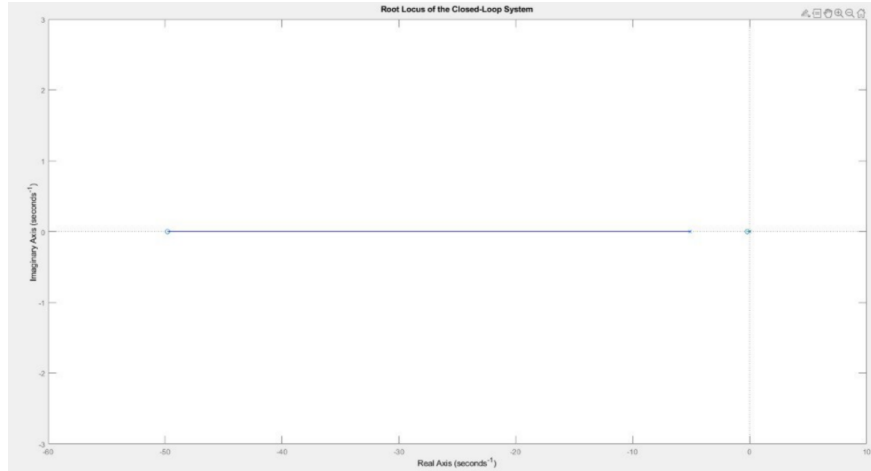


Figure 8: Root Locus Diagram

- The derivative gain K_d is found using $K_d = 8 \times K_p \times P_c$.

In the code, these calculated PID values are implemented to create a PID controller that is then used to control the system dynamics:

```
Km = 0.5;
tau = 0.2;
kthrust = 0.02;
m = 2.5;
g = 9.81;
s = tf('s');

G_motor = Km / (tau * s + 1);
G_thrust = kthrust;
H = G_motor * G_thrust / m;
Kc = 25;
Pc = 0.5;

Kp_pid = 0.6 * Kc;
Ki_pid = 2 * Kp_pid / Pc;
Kd_pid = Kp_pid * Pc / 8;
```

```

C_pid = pid(Kp_pid, Ki_pid, Kd_pid);

alpha_lead = 10;
beta_lead = 0.1;
C_lead = (s + alpha_lead) / (s + beta_lead);

sys_PID = feedback(C_pid * H, 1);
sys_lead = feedback(C_lead * H, 1);

figure;
subplot(2,2,1);
step(sys_PID);
title('Step Response with PID');

subplot(2,2,2);
step(sys_lead);
title('Step Response with Lead Compensator');

info_PID = stepinfo(sys_PID);
info_lead = stepinfo(sys_lead);

disp('Performance for PID:');
disp(info_PID);
disp('Performance for Lead Compensator:');
disp(info_lead);

```

Results:

They are similar, PID goes up to the axis a bit earlier thus more preferred.

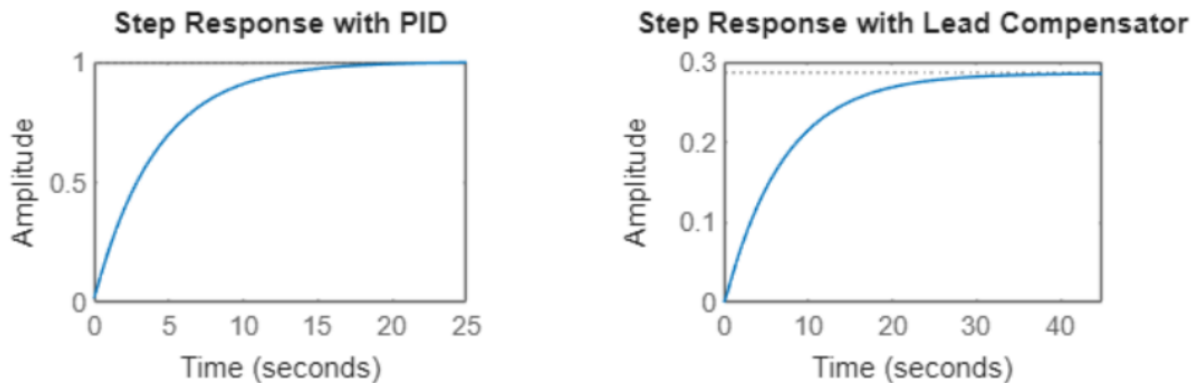


Figure 9: Step Response for PID and Lead Compensator

3.5 Interfaces

The drone I am building operates similarly to most other drones regarding internal interactions. The battery supplies power to the DC motors, while the electronic speed controllers (ESCs) regulate the speed of each motor, ensuring they operate at equal speeds and maintain stability.

What makes this drone unique is the integration of the Pixhawk 4, Raspberry Pi, and its components. These components are responsible for the autonomous functionality of the drone, allowing it to operate independently from a transmitter. The Raspberry Pi is the central computing unit, where all the necessary code is embedded, enabling the drone's autonomous control.

Additionally, the drone has a camera that continuously captures analog data. When the camera detects an ArUco marker, the onboard code processes the visual input and adjusts the drone's trajectory accordingly. This capability allows the drone to alter its path based on the marker's detection, further enhancing its autonomy.

3.5.1 ArUco Marker Detection

The auto marker detection works because the Raspberry Pi first needs the required libraries, one of which is the most important: OpenCV. OpenCV is a real-time image processing library that allows for the camera to read aruco and essentially allows the drone to eventually localize itself precisely on top of the truck like a landing pad; the rest of the

libraries will be listed below :

```
opencv-contrib-python (for ArUco marker detection)
numpy (for numerical operations)
picamera2 (for Raspberry Pi camera control)
libcamera (for camera support on newer Raspberry Pi OS versions)
python3-opencv (sometimes needed alongside opencv-contrib-python)
pip install dronekit pymavlink ( for localisation )
```

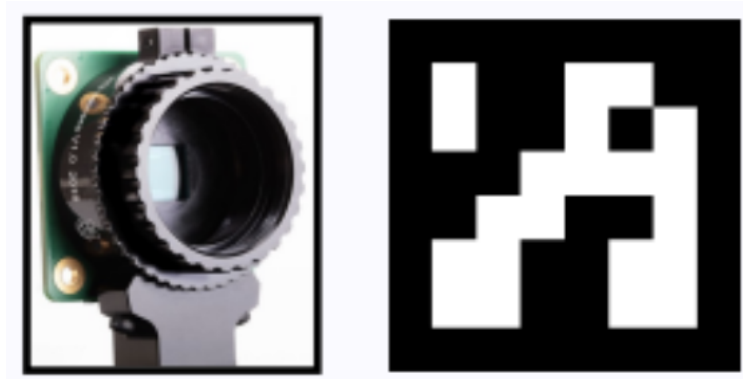


Figure 10: Raspberry Pi HQ Camera and ArUco Sample

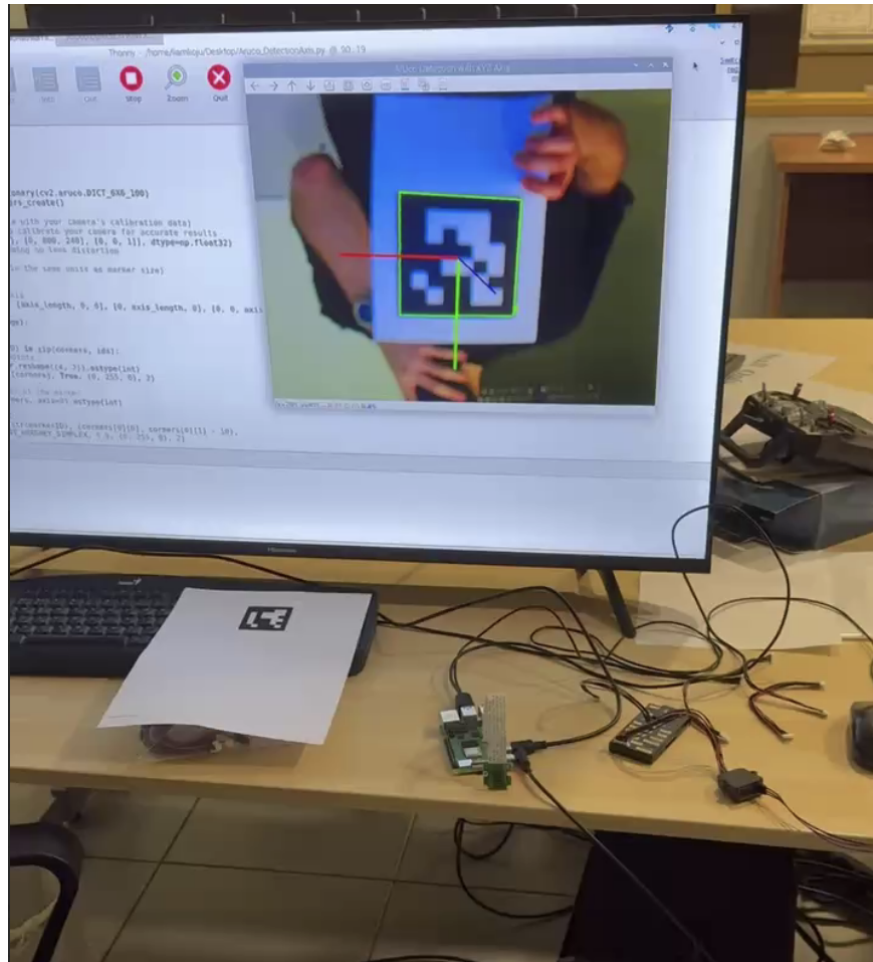


Figure 11: Real World Aruco Testing

Once the ArUco is confirmed to be working, we then apply it onto the drone, for the drone to use the ArUco as a landing pad for localization and to precisely land on any unique point within the warehouse rack, with the following code :

```
import cv2
import numpy as np
import time
from picamera2 import Picamera2

# Drone control imports (example using dronekit)
from dronekit import connect, VehicleMode, LocationGlobalRelative
from pymavlink import mavutil
```

```

# --- ArUco Setup ---
arucoDict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_100)
arucoParams = cv2.aruco.DetectorParameters_create()

# Camera calibration (adjust these values for your camera)
camera_matrix = np.array([[800, 0, 320], [0, 800, 240], [0, 0, 1]], dtype=np.float32)
dist_coeffs = np.zeros((4, 1))

# --- Drone Control Parameters ---
TARGET_ALTITUDE = 2.0 # meters
CENTER_THRESHOLD = 30 # pixels (how close to center before we consider it centered)
LAND_THRESHOLD = 0.15 # meters (how close vertically before landing)
DESCENT_RATE = 0.2 # m/s
MOVE_SPEED = 0.5 # m/s

# --- Drone Connection ---
def connect_drone():
    print("Connecting to drone...")
    vehicle = connect('/dev/ttyAMA0', wait_ready=True, baud=57600) # Adjust connection
    print("Drone connected!")
    return vehicle

def arm_and_takeoff(vehicle, altitude):
    print("Arming motors...")
    vehicle.mode = VehicleMode("GUIDED")
    vehicle.armed = True
    while not vehicle.armed:
        time.sleep(0.1)

```

```

print(f"Taking off to {altitude} meters...")
vehicle.simple_takeoff(altitude)

while True:
    if vehicle.location.global_relative_frame.alt >= altitude*0.95:
        break
    time.sleep(0.5)

# --- Control Logic ---
def center_drone_on_marker(vehicle, tvec, frame_center):
    """
    Adjust drone position to center over the marker
    tvec: translation vector from ArUco detection
    frame_center: (x,y) of image center
    """
    # Get marker position in camera frame
    marker_x, marker_y, marker_z = tvec[0][0]

    # Calculate errors from center
    error_x = frame_center[0] - marker_x
    error_y = frame_center[1] - marker_y

    # Determine movement commands
    if abs(error_x) > CENTER_THRESHOLD:
        if error_x > 0:
            # Move right
            send_ned_velocity(vehicle, 0, MOVE_SPEED, 0)
        else:
            # Move left
            send_ned_velocity(vehicle, 0, -MOVE_SPEED, 0)
    elif abs(error_y) > CENTER_THRESHOLD:

```

```

        if error_y > 0:
            # Move forward
            send_ned_velocity(vehicle, MOVE_SPEED, 0, 0)
        else:
            # Move backward
            send_ned_velocity(vehicle, -MOVE_SPEED, 0, 0)
    else:
        # Centered - descend if close enough
        if marker_z < LAND_THRESHOLD:
            print("Marker reached - landing!")
            vehicle.mode = VehicleMode("LAND")
        else:
            send_ned_velocity(vehicle, 0, 0, -DESCENT_RATE)

def send_ned_velocity(vehicle, vx, vy, vz):
    """
    Send velocity command to drone (NED coordinates)
    """
    msg = vehicle.message_factory.set_position_target_local_ned_encode(
        0,          # time_boot_ms
        0, 0,       # target system, target component
        mavutil.mavlink.MAV_FRAME_LOCAL_NED, # frame
        0b0000111111000111, # type_mask (only velocities enabled)
        0, 0, 0,    # x, y, z positions
        vx, vy, vz, # velocities
        0, 0, 0,    # acceleration
        0, 0)      # yaw, yaw_rate
    vehicle.send_mavlink(msg)

# --- Main Program ---

```

```

def main():
    # Connect to drone
    vehicle = connect_drone()

    # Start camera
    picam2 = Picamera2()
    config = picam2.create_preview_configuration(main={"size": (640, 480)})
    picam2.configure(config)
    picam2.start()
    time.sleep(2)

    # Get frame center
    frame_center = (320, 240) # Adjust if your resolution changes

    try:
        # Arm and takeoff
        arm_and_takeoff(vehicle, TARGET_ALTITUDE)

        while True:
            frame = picam2.capture_array()
            frame = cv2.cvtColor(frame, cv2.COLOR_RGBA2BGR)

            # Detect markers
            corners, ids, _ = cv2.aruco.detectMarkers(frame, arucoDict, parameters=aruco

            if ids is not None:
                for markerCorner, markerID in zip(corners, ids):
                    # Estimate pose
                    rvec, tvec, _ = cv2.aruco.estimatePoseSingleMarkers(
                        markerCorner, 0.05, camera_matrix, dist_coeffs)

```

```

        # Control drone
        center_drone_on_marker(vehicle, tvec, frame_center)

        # Visualization (optional)
        frame = aruco_display(corners, ids, frame)

    cv2.imshow("Drone Control", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

except KeyboardInterrupt:
    print("Landing drone...")
    vehicle.mode = VehicleMode("LAND")

finally:
    vehicle.close()
    picam2.stop()
    picam2.close()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

3.5.2 Servo Actuator Control

Unlike the Amazon drones, which rely on the servos themselves, which limits the amount of weight the package can be, however, in my system, that's not the case; the servo is inside a 3d-printed mechanism where the servo pushes a rod into two slots in which the entire physics is now engineering statics reliant, which is safer for the servo and more reliable in heavy payload holding, the below two images will be helpful in visualizing :

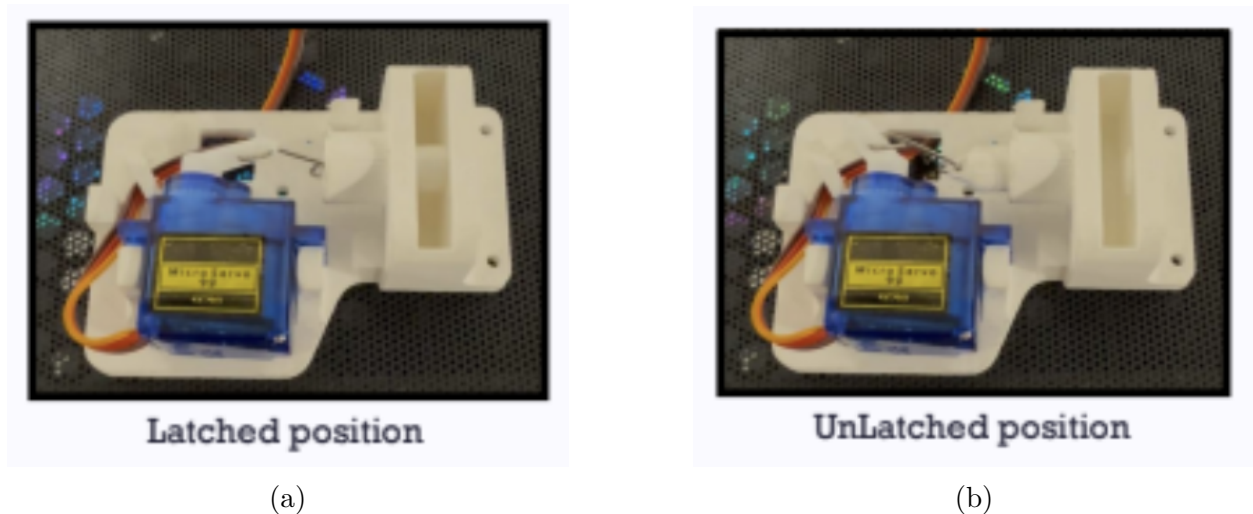


Figure 12: Operational states of the grappling mechanism

The way the servos work is simple: at the beginning of the operation, it triggers the servos, which latch onto the crate. Then the drone will navigate its way to its designated arco and land, and once it lands, the motors stop moving, and that's when it's safe for the payload to be safely dropped onto the landing pad. Thus, that's how the code works below :

```
from gpiozero.pins.pigpio import PiGPIOFactory
from gpiozero import AngularServo
import cv2
import numpy as np
import time
from picamera2 import Picamera2
from dronekit import connect, VehicleMode, LocationGlobalRelative
from pymavlink import mavutil

# --- Servo Setup ---
f = PiGPIOFactory()
right_servo = AngularServo(12, min_pulse_width=0.0006, max_pulse_width=0.0023, pin_factory=f)
left_servo = AngularServo(13, min_pulse_width=0.0006, max_pulse_width=0.0023, pin_factory=f)

def latch_crate():
```

```

        """Move servos to latch position (-90 and 90 degrees)"""
        right_servo.angle = -90
        left_servo.angle = 90
        print("Crate latched")

def unlatch_crate():
    """Return servos to neutral position (0 degrees)"""
    right_servo.angle = 0
    left_servo.angle = 0
    print("Crate unlatched")

# --- ArUco and Drone Setup ---
arucoDict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_100)
arucoParams = cv2.aruco.DetectorParameters_create()
camera_matrix = np.array([[800, 0, 320], [0, 800, 240], [0, 0, 1]], dtype=np.float32)
dist_coeffs = np.zeros((4, 1))

# Flight parameters
TARGET_ALTITUDE = 2.0 # meters
LAND_THRESHOLD = 0.2 # meters
CENTER_THRESHOLD = 30 # pixels
DESCENT_RATE = 0.2 # m/s
MOVE_SPEED = 0.5 # m/s

def connect_drone():
    print("Connecting to drone...")
    vehicle = connect('/dev/ttyAMA0', wait_ready=True, baud=57600)
    print("Drone connected!")
    return vehicle

```

```

def arm_and_takeoff(vehicle, altitude):
    print("Arming motors...")
    vehicle.mode = VehicleMode("GUIDED")
    vehicle.armed = True
    while not vehicle.armed:
        time.sleep(0.1)

    print(f"Taking off to {altitude} meters...")
    vehicle.simple_takeoff(altitude)
    while True:
        if vehicle.location.global_relative_frame.alt >= altitude*0.95:
            break
        time.sleep(0.5)

def center_drone_on_marker(vehicle, tvec, frame_center):
    marker_x, marker_y, marker_z = tvec[0][0]
    error_x = frame_center[0] - marker_x
    error_y = frame_center[1] - marker_y

    if abs(error_x) > CENTER_THRESHOLD:
        send_ned_velocity(vehicle, 0, MOVE_SPEED if error_x > 0 else -MOVE_SPEED, 0)
    elif abs(error_y) > CENTER_THRESHOLD:
        send_ned_velocity(vehicle, MOVE_SPEED if error_y > 0 else -MOVE_SPEED, 0, 0)
    else:
        if marker_z < LAND_THRESHOLD:
            print("Landing complete!")
            vehicle.mode = VehicleMode("LAND")
            while vehicle.armed: # Wait until landed
                time.sleep(0.5)
            unlatch_crate() # Unlatch when landed

```

```

        else:
            send_ned_velocity(vehicle, 0, 0, -DESCENT_RATE)

def send_ned_velocity(vehicle, vx, vy, vz):
    msg = vehicle.message_factory.set_position_target_local_ned_encode(
        0, 0, 0, mavutil.mavlink.MAV_FRAME_LOCAL_NED,
        0b0000111111000111, 0, 0, 0, vx, vy, vz, 0, 0, 0, 0, 0)
    vehicle.send_mavlink(msg)

def main():
    # Initial crate latching
    latch_crate()
    time.sleep(1) # Wait for servos to move

    # Connect to drone
    vehicle = connect_drone()

    # Start camera
    picam2 = Picamera2()
    config = picam2.create_preview_configuration(main={"size": (640, 480)})
    picam2.configure(config)
    picam2.start()
    time.sleep(2)

    frame_center = (320, 240)

    try:
        # Takeoff sequence
        arm_and_takeoff(vehicle, TARGET_ALTITUDE)

```

```

# Main control loop
while vehicle.armed: # Run while drone is flying
    frame = picam2.capture_array()
    frame = cv2.cvtColor(frame, cv2.COLOR_RGBA2BGR)

    corners, ids, _ = cv2.aruco.detectMarkers(frame, arucoDict, parameters=aruco

    if ids is not None:
        for markerCorner, markerID in zip(corners, ids):
            rvec, tvec, _ = cv2.aruco.estimatePoseSingleMarkers(
                markerCorner, 0.05, camera_matrix, dist_coeffs)
            center_drone_on_marker(vehicle, tvec, frame_center)

        cv2.imshow("Drone Control", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

except KeyboardInterrupt:
    print("Emergency landing!")
    vehicle.mode = VehicleMode("LAND")

finally:
    vehicle.close()
    picam2.stop()
    picam2.close()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

3.6 Modes of Operations

The drone goes through three modes of operation: Altitude Hold mode, ATTI mode, and Return to Home (RTH) mode.

In Altitude Hold mode, the drone maintains a constant altitude while performing its tasks. The drone undergoes different stages when picking up and placing a payload. Initially, it flies at a specified altitude, then changes its position while maintaining its direction.

The drone operates in ATTI mode (Attitude mode) to maintain its direction during the flight. In this mode, the onboard sensors and gyroscopes control the drone, ensuring stability while adjusting the direction.

Once the payload is dropped at its destination, the drone enters the RTH (Return to Home) mode. This mode ensures that the drone automatically returns to its initial position, completing its mission.

Since the Drone is in a mode where it makes sure everything is slow and steady, the drone should not tilt, or else it would damage the contents within the payload, ensuring it stays almost entirely verticle as it cruses in the air.



Figure 13: Drone maintains its vertical posture as it glides while also maintaining its pose estimation with respect to the ArUco landing pad

3.7 Proposed Capabilities

The drone's path depends on the Aruco markers, which can be printed on pieces of paper, their placement, and the code given to the drone, instructing it on how to react when detecting the Aruco markers with its camera.

It is possible to have two buttons and two Aruco markers. If coded accordingly, the drone can choose one of the two locations to drop off the payload. This feature gives the project more potential for future development and expansion.

The drone's speed will be adjusted to maintain a slow and steady pace. This ensures that the ultrasonic sensor can react in time and prevent a collision if the drone starts to bump into an obstacle.

3.8 Methodology

Other members of my facility are working on a drone simulation with aspects similar to my project. Thus, I can cooperate with that group to have a deeper understanding and a safer approach to start practically building on my project.

The steps I will take in doing my project are first, building the drone, connecting all the components using the soldering kit, studying the payload and drone and making sure their sizes match, and making sure I place the servo motor at the appropriate point on the frame of the drone, leveling well with the payload, and ensuring it grips onto it.

Then, I will proceed with the software part and trial and error to see if it makes the drone steady. I will place the Aruco markers in the exact location where the payload must be placed so the drone and the code cooperatively read the marker and execute the action.

I will study the aerodynamics of the drone and its torque to observe its weight-holding capabilities. I will test the drone with different payload weights to see how well the control system works and if it handles the loads.

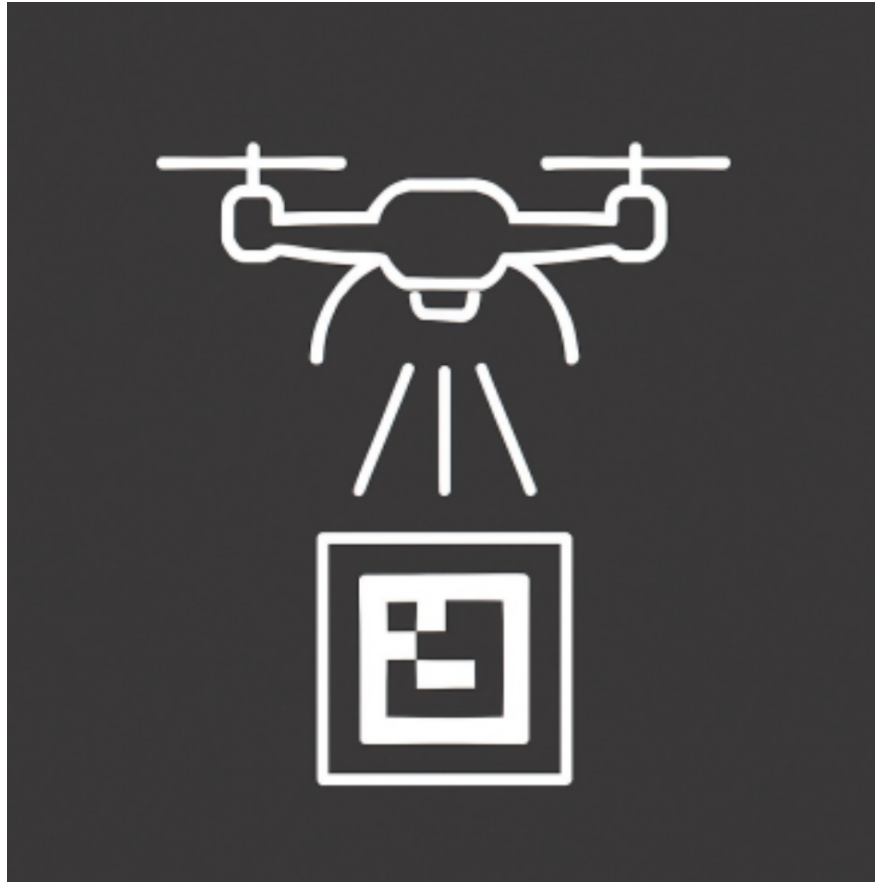


Figure 14: Drone-Aruco Detection Illustration

4 Physical Environment

The environment for the drone is supposed to be an indoor warehouse where boxes are being moved around. It will have a ceiling; therefore, ensuring the altitude does not exceed a certain point is important.

Since it is indoors, it has disadvantages. However, the advantage is that no moody weather, like rain or heat, interferes with the system. That would mean that Aruco markers must be placed around the areas where the loads are to be placed, as the GPS module will not function properly in an enclosed space.

5 Support Environment

The warehouse needs a support environment where additional implementations must be made inside the pre-existing warehouse; it needs landing pads for each designated location for the drone to understand where to place any crate specifically. Similarly to a market where every product has barcodes, the warehouse racks are required to be rebuilt in a way where there are unique Aruco markers in every space within the surfaces of the shelves, ensuring that whatever ArUco ID you give the drone to go and drop off the crate, it will find it, thus playing a crucial role in the pre-designed environment for the drone. Another supporting environment that may be required is that the racks may need to be designed in a way where there is most space in the middle sections for the drone with the payload attached to fit in easily and glide through without having to bump into the poles of the shelves and be able to place payload anywhere throughout the rack successfully. Finally, an extension for the crates is required for there to be a hole so that the reliable gripping mechanism can fit through.

6 Risks and Potential Issues

There are some risks for the early stages of this project; for example, once the signal is lost for the drone and it takes off and goes anywhere, it will be dangerous in a warehouse environment in the sense that there is a heavily spinning propellor within an enclosed area and that can cause heavy damage on anyone that gets into contact with this. Also, the size of the drone prototype might be too big for now to fit into the middle sections of the warehouse racks, but with future developments and making the drone smaller in size, that risk could be bypassed. With enough regulation, many of these risks can be avoided in the long run. For example, if a payload may be too heavy, an implementation of synchronizing drones can help aid the thrust and thus lift an excessively heavy crate, fixing more and more problems in the future.

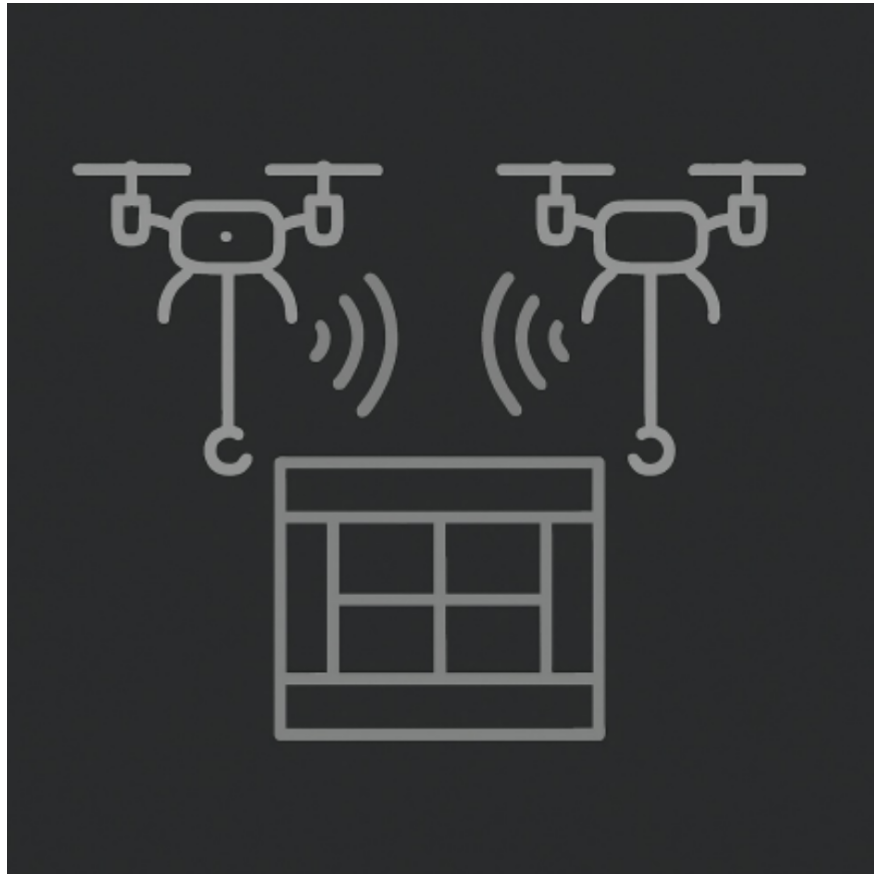


Figure 15: Illustration of Multiple Drones synchronized to carry heavier loads, for future studies